SAST EXECUTIVE REPORT

Static Application Security Testing Analysis

Repository: my-app
Scan Date: 2026-06-21
Report Type: CNES-Style Executive Summary

3 CRITICAL	1 HIGH	3 MEDIUM	0 LOW

Executive Summary

7
TOTAL FINDINGS

7
ACTIONABLE ITEMS
(WARNING/ERROR)

Risk Distribution

CRITICAL	HIGH RISK
3	1

MEDIUM RISK	LOW RISK
3	0

Methodology: This report presents findings from automated Static Application Security Testing (SAST) analysis. Only WARNING and ERROR severity findings are included in the detailed table below, as INFO-level findings do not require immediate action. Severity levels (CRITICAL/HIGH/MEDIUM/LOW) are determined based on impact and likelihood metrics. Findings have been categorized according to CWE and OWASP standards.

Detailed Findings (WARNING/ERROR Only)

SEVERITY	FILE LOCATION	RULE ID	DESCRIPTION	SECURITY STANDARDS
MEDIUM	report.py Line 423	python.flask.security.xss.audit.direct-use-of-jinja2.direct-use-of-jinja2	Detected direct use of jinja2. If not done properly, this may bypass HTML escaping which opens up the application to cross-site scripting (XSS) vulnerabilities. Prefer using the Flask method 'render_template()' and templates with a '.html' extension in order to prevent XSS.	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') flask
MEDIUM	web/app.py Line 387	python.flask.security.xss.audit.direct-use-of-jinja2.direct-use-of-jinja2	Detected direct use of jinja2. If not done properly, this may bypass HTML escaping which opens up the application to cross-site scripting (XSS) vulnerabilities. Prefer using the Flask method 'render_template()' and templates with a '.html' extension in order to prevent XSS.	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') flask
CRITICAL	web/app.py Line 408	python.flask.security.open-redirect.open-redirect	Data from request is passed to redirect(). This is an open redirect and could be exploited. Consider using 'url_for()' to generate links to known locations. If you must use a URL to unknown pages, consider using 'urlparse()' or similar and checking if the 'netloc' property is the same as your site's host name. See the references for more information.	CWE-601: URL Redirection to Untrusted Site ('Open Redirect') flask
CRITICAL	web/app.py Line 411	python.flask.security.open-redirect.open-redirect	Data from request is passed to redirect(). This is an open redirect and could be exploited. Consider using 'url_for()' to generate links to known	CWE-601: URL Redirection to Untrusted Site ('Open Redirect') flask

SEVERITY	FILE LOCATION	RULE ID	DESCRIPTION	SECURITY STANDARDS
			locations. If you must use a URL to unknown pages, consider using 'urlparse()' or similar and checking if the 'netloc' property is the same as your site's host name. See the references for more information.	
CRITICAL	web/app.py Line 428	python.flask.security.open-redirect.open-redirect	Data from request is passed to redirect(). This is an open redirect and could be exploited. Consider using 'url_for()' to generate links to known locations. If you must use a URL to unknown pages, consider using 'urlparse()' or similar and checking if the 'netloc' property is the same as your site's host name. See the references for more information.	CWE-601: URL Redirection to Untrusted Site ('Open Redirect') flask
HIGH	web/app.py Line 474	python.flask.security.audit.app-run-param-config.avoid_app_run_with_bad_host	Running flask app with host 0.0.0.0 could expose the server publicly.	CWE-668: Exposure of Resource to Wrong Sphere flask
MEDIUM	web/templates/index.html Line 234	python.django.security.django-no-csrf-token.django-no-csrf-token	Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.	C W django